

# ERD

Fyzická struktura databáze odvozená z doménového modelu

Zatímco doménový model popisuje, jak o problému přemýšlím, tento diagram ukazuje, jak se data fyzicky ukládají. Model je rozdělený do tří částí, aby zůstal čitelný – jádro reklamace, snapshoty dat z cizího systému a konfigurace tenanta.

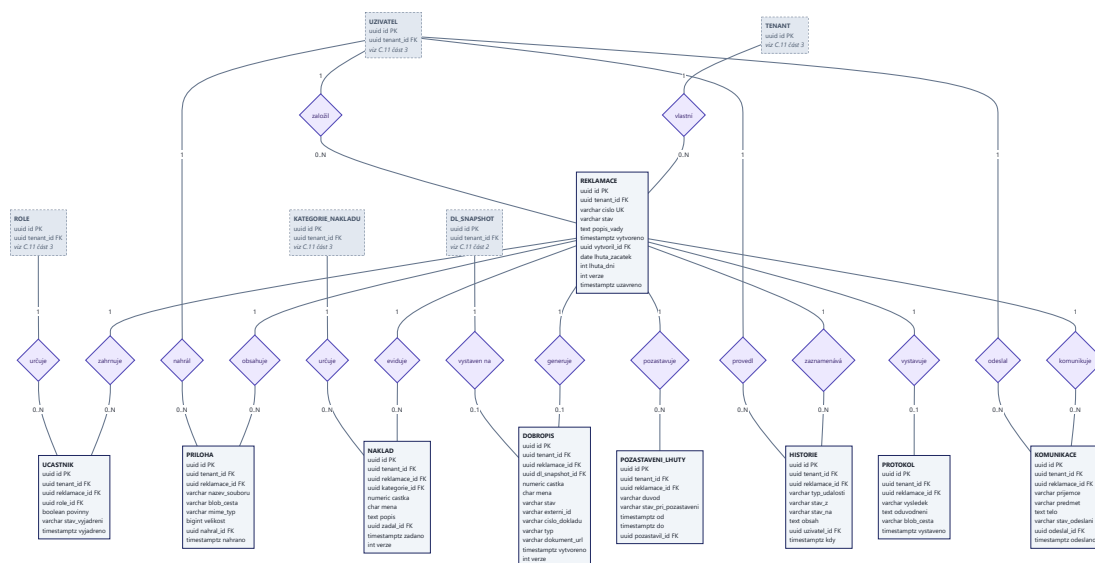
**Notace: entity, relační typ a kardinalita zvlášť (Chen).** Diagramy používají Chenovu notaci – obdélník je entita, kosočtverec je vztah mezi entitami a číslo na spojnici ( **1** , **N** , případně **0..1** nebo **1..N** u volitelných či povinných vazeb) je kardinalita na té straně. Vztah je tak vlastní uzel diagramu, ne jen popisek na čáře.

**Zjednodušení oproti čisté Chenově notaci:** atributy zůstávají vypsané uvnitř obdélníku entity, ne jako samostatné elipsy spojené čarou. Při desítkách sloupců na tabulku by elipsy diagram znečitelnily; seznam atributů uvnitř entity nese stejnou informaci kompaktněji.

Přerušovaně orámovaná entita je definovaná v jiné části modelu – zobrazuje jen primární a cizí klíče, plný výčet sloupců je u sekce, na kterou odkazuje.

**Rozsah kardinality je vždy vypsaný celý** – **0..N** , **1..N** nebo **0..1** , nikdy jen holé **N** . Většina vztahů v jádru je volitelná ( **0..N** ) – reklamace může vzniknout bez nákladů, příloh či komunikace. Výjimkou je snapshot dodávky: dodací list musí obsahovat aspoň jednu recepturu ( **1..N** ), záměsový protokol aspoň jednu navážku a receptura aspoň jednu položku složení – to jsou povinné části dat, které bez nich nedávají smysl, a liší se tak od certifikátů či laboratorních výsledků, jež zůstávají volitelné ( **0..N** ).

## 1. Jádro reklamace



**Lhůta je rozložená do sloupců, pozastavení má vlastní tabulku.** `Lhuta` je v doménovém modelu hodnotový objekt bez vlastní identity, proto se ukládá jako sloupce nadřazené tabulky – začátek a počet dnů.

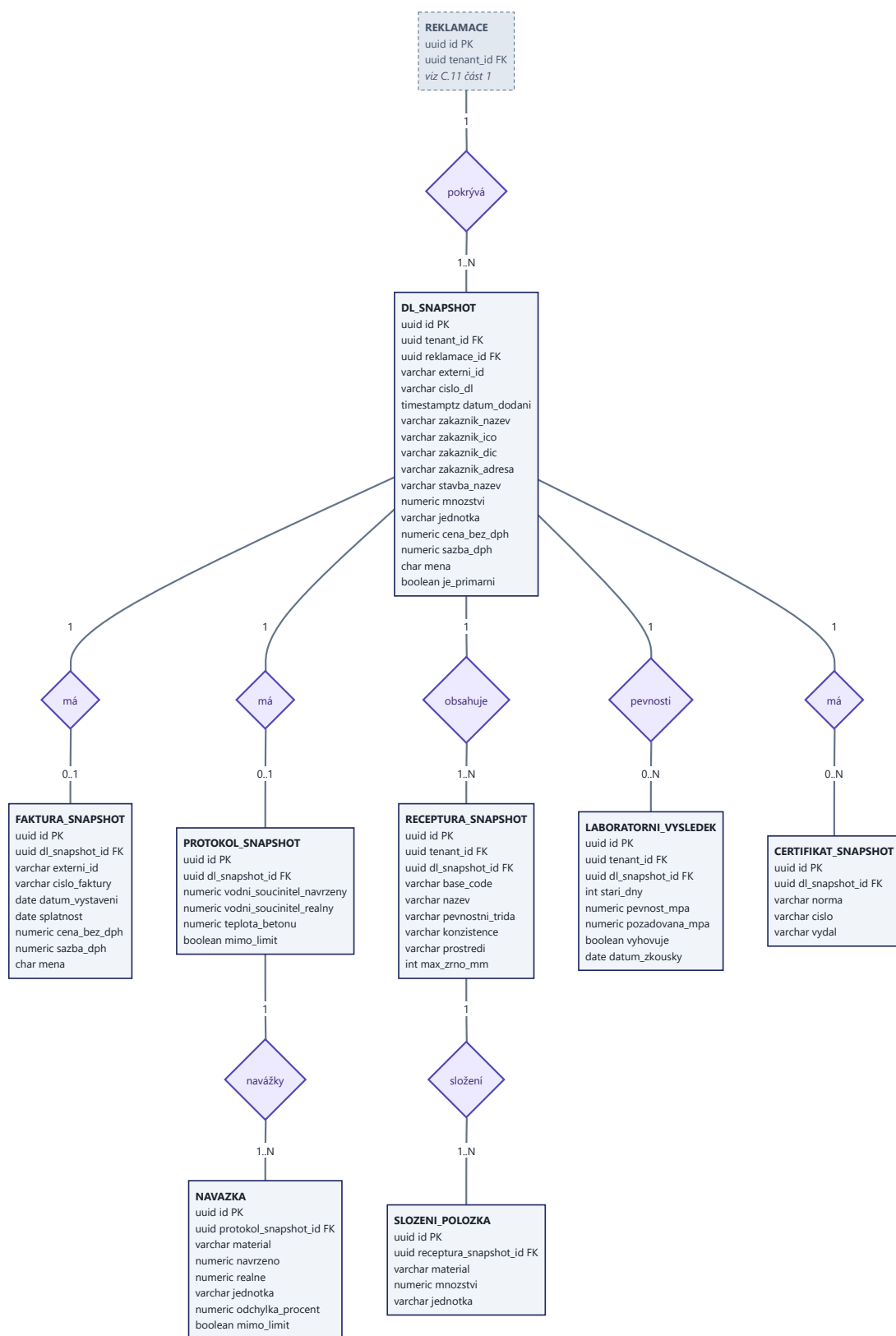
Jednotlivá pozastavení ale vlastní tabulku mají. Původně stačil jediný sloupec se součtem dnů strávených v pozastaveném stavu, jenže z něj nešlo zjistit, *kdy* a *jak dlouho* byla lhůta zastavená a z jakého důvodu. U lhůty, která plyne ze smlouvy a v případě sporu se dokazuje, je tahle informace podstatná – tabulka `pozastaveni_lhuty` proto eviduje každý interval zvlášť, včetně stavu, ve kterém k němu došlo, a uživatele, který ho vyvolal.

Zbývající lhůta se pak počítá jako `lhuta_dni` minus součet uzavřených intervalů. Právě probíhající pozastavení pozná podle prázdného sloupce `do`.

**Dobropis má sloupec `externi_id`, který je zpočátku prázdný.** Doklad vzniká v cizím systému asynchronně. Do té doby je ve stavu `Ceka` a odkaz na skutečný doklad chybí – to je právě ta informace, kterou potřebuje uživatel vidět v detailu reklamace.

## 2. Snapshoty dat z cizího systému

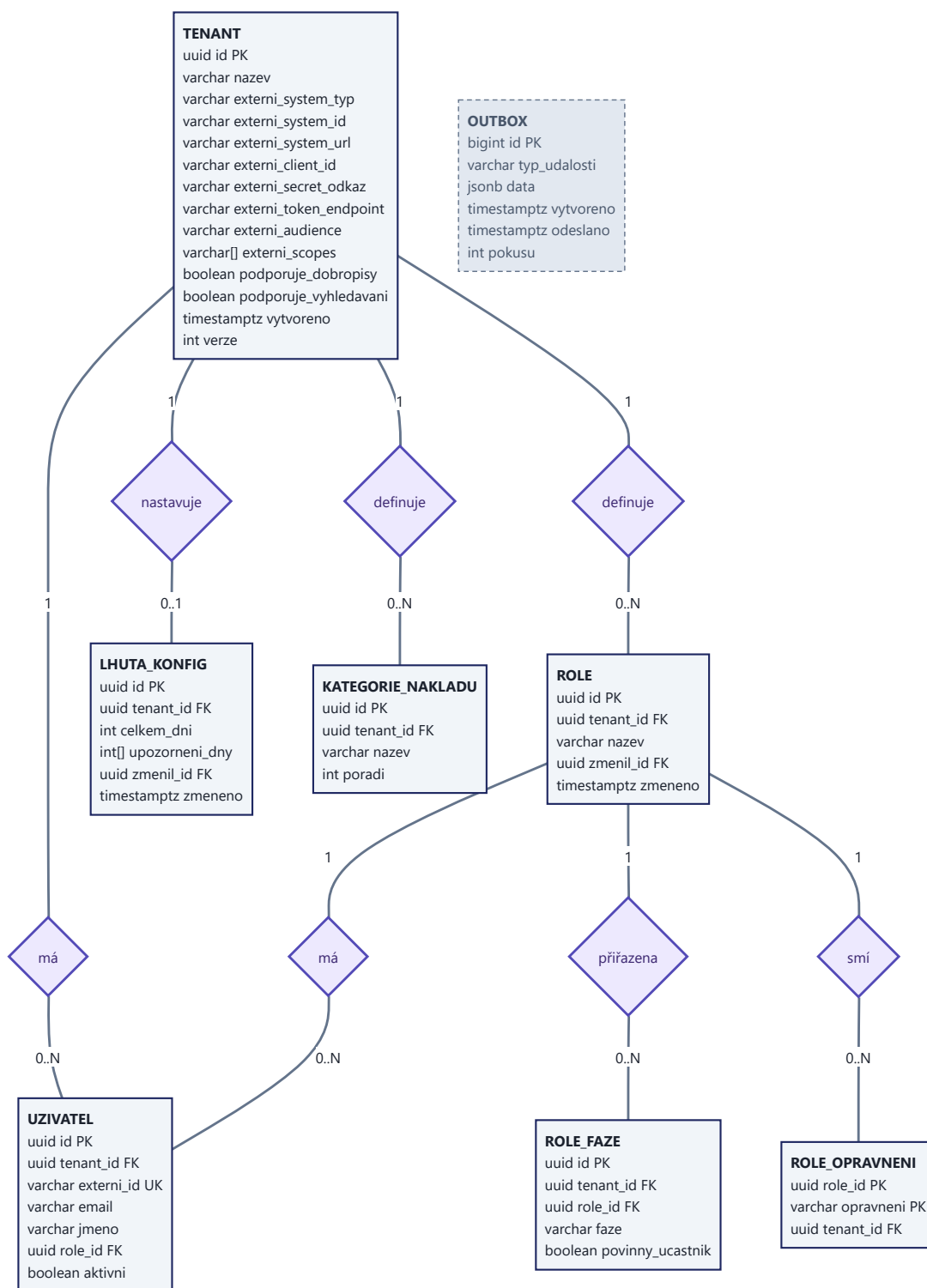
Kopie dat pořízená při zakládání reklamace. Po vytvoření se už nemění – proto v těchto tabulkách nejsou sloupce pro úpravu ani odkazy na uživatele.



**Proč jsou snapshoty vlastní tabulky, a ne jen odkaz do cizího systému.** Reklamáce je právní dokument o stavu v konkrétním okamžiku. Kdyby se data načítala z cizího systému až při zobrazení, změnil by pozdější zásah do dokladu i obsah reklamáce. Vlastní kopie také znamená, že reklamáce

zůstává čitelná, i když je cizí systém nedostupný, a že systém funguje pro tenanty bez jakékoli integrace.

### **3. Konfigurace a infrastruktura**



**Kategorie nákladů jsou číselník, ne volný text.** Tabulka `naklad` odkazuje na `kategorie_nakladu` cizím klíčem. Referent tak vybírá z připraveného seznamu, který si tenant nastavuje sám – bez toho by se v datech objevilo „laboratoř“, „laborator“ i „lab. zkouška“ a součty podle kategorií by ztratily smysl.

**Konfigurovatelnost rolí nese trojice tabulek.** `ROLE` nese jen název – co role smí, je v `ROLE_OPRAVNENI`, kde se role účastní, v `ROLE_FAZE`. Rozdělení odpovídá tomu, že jde o dvě nezávislé věci: oprávnění říká, jaké akce uživatel může provést, přiřazení k fázi říká, kdy se od něj čeká vyjádření.

Samotné fáze a povolené přechody mezi nimi jsou pevné – konfigurovatelnost se týká jen toho, kdo se které fáze účastní a co smí, nikoli pravidel workflow. To je jádro výzkumné otázky: tenant mění obsazení rolí, ne strukturu procesu.

**Oprávnění jako řádky, ne jako dokument.** Původně byla oprávnění uložena jako netypovaný JSON ve sloupci `ROLE.opravneni`. Databáze tak neměla jak ověřit, že hodnota dává smysl, a nešlo se dotázat, které role smějí konkrétní akci – to by vyžadovalo prohledání dokumentů všech řádků.

Tabulka `ROLE_OPRAVNENI` má složený primární klíč (`role_id`, `opravneni`), který sám o sobě brání přiřazení téhož oprávnění dvakrát. Vlastní `id` nemá, protože nenese žádný další údaj – na rozdíl od `ROLE_FAZE`, kde přibývá příznak povinného účastníka.

Hodnoty oprávnění pocházejí z výčtu definovaného v kódu. Nové oprávnění nemůže vzniknout samotným zápisem do databáze – vždy k němu musí existovat místo v aplikaci, které ho kontroluje.

**Typované pole místo dokumentu.** Sloupec `upozorneni_dny` obsahuje prostý seznam čísel – kolik dní před koncem lhůty se má poslat upozornění, například `{5, 2, 0}`. Typ `int[]` zaručí, že se do něj nedostane nic jiného než celá čísla; `jsonb`, který tu byl původně, by přijal libovolný obsah.

Samostatná tabulka by sem nepatřila. Hodnoty se nikdy nečtou jednotlivě – načítají se vždy celé spolu s konfigurací tenanta, takže by přidala spojení bez užitku.

Tabulka `OUTBOX` stojí mimo relace záměrně: je to infrastruktura, ne doménová data.

**Kdo co udělal.** Sloupec s odkazem na uživatele nesou všechny záznamy, u kterých je zpětně podstatné dohledat autora – kdo reklamaci založil, nahrál přílohu, odeslal zprávu nebo zadal náklad. Totéž platí pro konfiguraci: změna délky lhůty ovlivní vyřizování všech budoucích reklamací, takže musí být dohledatelné, kdo ji provedl a kdy.

Snapshoty naopak žádný takový sloupec nemají. Nevytváří je uživatel, ale systém při zakládání reklamace – a okamžik jejich pořízení odpovídá vzniku reklamace.

**Sloupec `verze` chrání před tichým přepsáním změn.** Reklamaci může mít otevřenou víc lidí zároveň – referent doplňuje náklady, vedoucí mění stav. Bez ochrany by poslední zápis přepsal ten předchozí a nikdo by se to nedozvěděl.

Při každé změně se hodnota zvýší a podmínkou zápisu je shoda s hodnotou, kterou klient načetl. Když se mezitím změnila, zápis neprojde a aplikace vrátí `409` – uživatel uvidí, že data mezitím někdo upravil, místo aby o svou změnu tiše přišel.

Sloupec nesou tabulky, se kterými pracuje víc lidí nezávisle: `reklamace`, `naklad`, `dobropis` a `tenant`. Snapshots a záznamy historie ho nemají – vznikají jednorázově a nikdy se neupravují.

**Tenant nese konfiguraci napojení na svůj systém.** Reklamační aplikace nemá adresu cizího systému zadanou v kódu – skládá ji z `externi_system_url` a cesty z kontraktu. Díky tomu funguje stejný kód pro libovolného zákazníka bez ohledu na to, jaký systém provozuje.

Příznaky `podporuje_dobropisy` a `podporuje_vyhledavani` říkají, kterou úroveň integrace daný tenant implementoval. Bez nich by aplikace posílala požadavky na endpointy, které u části zákazníků neexistují – zprávy by se opakovaly, končily v dead-letter frontě a vedoucí by dostával upozornění na chybu, která žádná není.

Přihlašovací tajemství v databázi neleží. Sloupec `externi_secret_odkaz` obsahuje jen identifikátor záznamu v Azure Key Vaultu, odkud si aplikace hodnotu vyzvedne za běhu.

**Autentizace není u všech tenantů stejná.** U ICOBIS se token vydává proti firemnímu IdP se scopey `icobis.read` a `icobis.write`, jiný systém má vlastní autorizační server, vlastní publikum i vlastní názvy oprávnění. Aplikace proto nemá autentizační parametry zadrátované – `externi_token_endpoint`, `externi_audience` a `externi_scopes` jsou součástí konfigurace tenanta stejně jako adresa systému.

## Multi-tenancy

Data všech tenantů leží v jedné databázi. Oddělení zajišťuje sloupec `tenant_id` doplněný o zabezpečení na úrovni řádků.

```
-- tenant_id nese i tabulky, které by ho mohly dědit přes vazbu
ALTER TABLE dl_snapshot ADD COLUMN tenant_id uuid NOT NULL;

-- databáze sama odfiltruje cizí řádky, i kdyby dotaz podmínku vynechal
ALTER TABLE reklamace ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_izolace ON reklamace
    USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

**Kde `tenant_id` je a kde ne.** Sloupec nesou všechny tabulky, na které se dotazují přímo – `reklamace`, její přímé části a snapshoty dodacího listu a receptury. Zabezpečení na úrovni řádků potřebuje sloupec v samotné tabulce, a dotazy se tím zbaví zbytečných spojení přes rodiče.

Tabulky, které se načítají výhradně spolu se svým rodičem, ho nemají – [navazka](#) , [slozeni\\_polozka](#) , [certifikat\\_snapshot](#) , [faktura\\_snapshot](#) a [protokol\\_snapshot](#) . Nikdy se na ně neptám samostatně, takže je chrání politika nadřazené tabulky. Denormalizace tak zůstává omezená na místa, kde skutečně něco přináší.

## Indexy

| Index                                                  | Proč                                                                                                                  |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>reklamace (tenant_id, stav)</code>               | Seznam reklamací filtrovaný podle stavu – nejčastější dotaz aplikace                                                  |
| <code>reklamace (tenant_id, cislo)</code> unikátní     | Číslo reklamace je jedinečné v rámci tenanta, ne globálně                                                             |
| <code>receptura_snapshot (tenant_id, base_code)</code> | Statistika reklamací pro recepturu, kterou volá cizí systém při zobrazení detailu                                     |
| <code>dl_snapshot (tenant_id, externi_id)</code>       | Kontrola, zda už na daný doklad reklamace existuje                                                                    |
| <code>role_opravneni (tenant_id, opraveni)</code>      | Opačný dotaz než složený klíč – které role smějí danou akci. Potřeba při zobrazení, kdo může reklamaci uznat          |
| <code>pozastaveni_lhuty (reklamace_id, do)</code>      | Výpočet zbývajících lhůt – potřebuje uzavřené intervaly i to, zda některý právě probíhá                               |
| <code>outbox (odeslano)</code> částečný                | Výběr neodeslaných událostí. Částečný index přes <code>WHERE odeslano IS NULL</code> zůstává malý i při velkém objemu |

## Vztah k doménovému modelu

| Doménový model           | ERD                                                         | Proč                                             |
|--------------------------|-------------------------------------------------------------|--------------------------------------------------|
| Reklamace (agregát)      | tabulka <code>reklamace</code>                              | Kořen agregátu je vlastní tabulka                |
| Stav (hodnotový objekt)  | sloupec <code>stav</code>                                   | Bez identity, ukládá se jako hodnota             |
| Lhuta (hodnotový objekt) | sloupce <code>lhuta_zacatek</code> , <code>lhuta_dni</code> | Několik souvisejících hodnot v nadřazené tabulce |



|                           |                                                 |                                                       |
|---------------------------|-------------------------------------------------|-------------------------------------------------------|
| PozastaveniLhuty (entita) | tabulka <code>pozastaveni_lhuty</code>          | Má identitu a je jich víc – každý interval samostatně |
| Naklad (entita)           | tabulka <code>naklad</code>                     | Má identitu a je jich víc                             |
| Castka (hodnotový objekt) | sloupce <code>castka</code> , <code>mena</code> | Dvojice hodnot, ne samostatná entita                  |

**Pravidlo, podle kterého se to převádí:** entita se stane tabulkou, hodnotový objekt sloupcem nebo skupinou sloupců. Když je hodnotových objektů v kolekci víc, stane se z nich tabulka bez vlastního významu mimo rodiče – jako `navazka` nebo `slozeni_polozka` .